

Report conversation

I have some duplication in my TypeScript code. I resolve it, I want to create a discriminated union based on the keys and values of the interface. My code is blow. Is it possible to do what I want?

```
type PrefixMap = {
  nprofile: ProfilePointer
  nrelay: string
  nevent: EventPointer
  naddr: AddressPointer
  nsec: string
  npub: string
  note: string
}

type DecodeValue<Prefix extends keyof PrefixMap, Data>
= {
  type: Prefix
  data: Data
}

export type DecodeResult =
  | DecodeValue<'nprofile', ProfilePointer>
  | DecodeValue<'nrelay', string>
  | DecodeValue<'nevent', EventPointer>
  | DecodeValue<'naddr', AddressPointer>
  | DecodeValue<'nsec', string>
  | DecodeValue<'npub', string>
  | DecodeValue<'note', string>
```

Yes, you can simplify this code by leveraging TypeScript's Mapped Types and Lookup Types to create your Discriminated Union in a more concise way. Here's how you can do it:

typescript

 Copy code

```
type PrefixMap = {  
  nprofile: ProfilePointer  
  nrelay: string  
  nevent: EventPointer  
  naddr: AddressPointer  
  nsec: string  
  npub: string  
  note: string  
}  
  
type DecodeValue<Prefix extends keyof PrefixMap> = {  
  type: Prefix  
  data: PrefixMap[Prefix]  
}  
  
export type DecodeResult = {  
  [P in keyof PrefixMap]: DecodeValue<P>  
}[keyof PrefixMap]
```

This should give you the equivalent result to what you had, but with less repetition. Here, `DecodeValue` has been redefined to automatically infer the correct type of `data` based on the `type` key.

The `DecodeResult` type is then created by mapping `DecodeValue` over each key of `PrefixMap` and creating a union of the results.

Please note that you must ensure that you're using TypeScript 2.1 or later because Mapped Types were introduced in TypeScript 2.1.